

Unit 02: Software process models

CONTENTS

Objectives

Introduction

2.1 Processes and Models

2.2 The Software Process Model

2.3 Characteristics of a Software Model

2.4 Software Development Life Cycle

2.5 Prototype model

2.6 Water Fall Model

2.7 V model

2.8 Incremental Model

2.9 Agile method of software development

Summary

Keywords

Self Assessment

Answer for Self Assessment

Review Questions

Further Reading

Objectives

After studying this unit, you will be able to:

- Discuss the concept of Processes and Models
- Explain the Software Process Model
- Characteristics of a Software Model
- Describe various process models
- Agile method of software development

Introduction

The concept of process is central to the software engineering method. "A particular technique of doing something, usually requiring a number of steps or operations," defines a process. The term "software process" refers to the way of developing software in software engineering. A software process is a set of operations that must be completed in order to create high-quality software. Is it possible to refer to the process as software engineering? Yes and no are the answers. The term "process" refers to the methods used in the development of software. Also included in the process are the technical procedures and tools used in software engineering. Software must be developed keeping in mind the demands of the end-user using a defined software process. In this unit, we will discuss the concept of software process models. We will also discuss different types of process models such as waterfall model, iterative model, etc.

2.1 Processes and Models

The software process describes how to manage and organise a software development project while taking limits and restrictions into account. A software process is a collection of actions linked by ordering constraints, with the goal of producing the intended output if the activities are completed

correctly and in compliance with the ordering constraints. The goal is to produce high-quality software at a reasonable cost. Clearly, a method is not acceptable if it does not scale up and cannot handle massive software projects or produce high-quality software.

Many processes are usually running at the same time in large software development organisations. Many of these have little to do with software engineering, yet they do have an impact on software development. These process models could be classified as non-software engineering. Processes that fall under this include business process models, social process models, and training models. These processes have an impact on software development, although they are outside the scope of software engineering.

The process that deals with the technical and management issues of software development is called a software process. Clearly, many different types of activities need to be performed to develop software

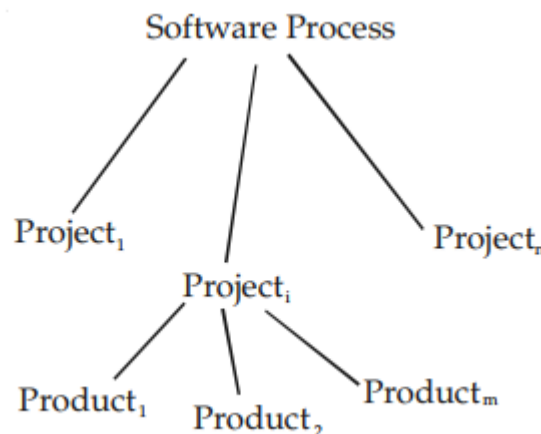
Because different types of activities are usually performed by different persons, it is better to think of the software process as a collection of component processes, each of which has a specific type of activity. Each of these component processes usually has a different goal in mind, while they obviously work together to achieve the overall software engineering goal.

Software Process framework is a set of guidelines, concepts and best practices that describes high level processes in software engineering. It does not talk about how these processes are carried out and in what order.

As previously stated, a software process defines a method for building software. A software project, on the other hand, is a development project that involves the usage of a software process. A software project's outputs are known as software products. Each software development project begins with a set of requirements and is expected to end with software that meets those requirements. A software process is an abstract set of operations that must be carried out in order to get from user requirements to the end result.

One can view the software process as an abstract type, and each project is done using that process as an instance of this type. In other words, there can be many projects for a process and there can be many products produced in a project. This relationship is shown in Figure

Relation between Processes, Projects and Products



The sequence of activities specified by the process is typically at an abstract level because they have to be usable for a wide range of projects. Hence, “implementing” them in a project is not straightforward.

Example: Let us take the example of book writing. A process for book writing on a subject will be something like this:

1. Set objectives of the book – audience, marketing etc.
2. Outline the contents.
3. Research the topics covered.
4. Do literature survey.

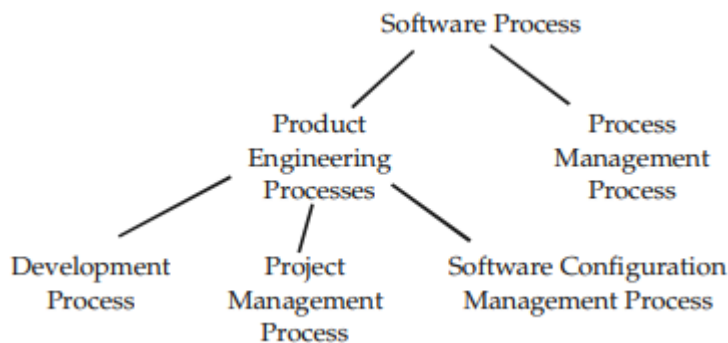
5. Write and/or compile the content.
6. Get the content evaluated by a team of experts.
7. Proof read the book.
8. Make corrections, if any.
9. Get the book typeset.
10. Print the book.
11. Bind the book.

Overall, the process specifies actions that are not project-specific at an abstract level. It's a generalised list of tasks that doesn't provide a precise roadmap for a specific project. The detailed roadmap for a project is the project plan that outlines what precise actions to undertake for this project, when to do them, and how to keep the project on track. The project plan for publishing a book on Software Engineering, in our example, will be a comprehensive marked map outlining the activities, as well as additional specifics such as plans for getting illustrations, photographs, and so on.

It should be obvious that a project is driven by its process. A method restricts a project's degrees of freedom by stating which activities must be completed and in what order. Furthermore, the project plan, which is itself within the process's bounds, specifies restrictions on the degrees of freedom for a specific project. To put it another way, a project plan cannot include an action that is not part of the process.

Because each project is an instance of the process it follows, it is fundamentally the process that determines the expected outcomes of a project. As a result, software engineering places a strong emphasis on the process.

Software Process



A software process is the set of activities and associated results that produce a software product. Software engineers mostly carry out these activities. There are four fundamental process activities, which are common to all software processes. These activities are:

1. Software Specification: The functionality of the software and constraints on its operation must be defined.
2. Software Development: The software to meet the specification must be produced.
3. Software Validation: The software must be validated to ensure that it does what the customer wants.
4. Software Evolution: The software must evolve to meet changing customer needs. Different software processes organize these activities in different ways and are described at different levels of detail. The timing of the activities varies as does the results of each activity.

Different organizations may use different processes to produce the same type of product. However, some processes are more suitable than others for some types of application. If an inappropriate process is used, this will probably reduce the quality or the usefulness of the software product to be developed.

Different software processes organize these activities in different ways and are described at different levels of detail. The timing of the activities varies as does the results of each activity. Different organizations may use different processes to produce the same type of product. However,

some processes are more suitable than others for some types of application. If an inappropriate process is used, this will probably reduce the quality or the usefulness of the software product to be developed.

2.2 The Software Process Model

A software process model is a simplified representation of a software process provided from a certain viewpoint. Because models are by definition simplifications, a software process model is an abstraction of the process being represented. Activities that are part of the software process, software products, and the responsibilities of persons involved in software engineering can all be included in process models.

Example: Some examples of the types of software process model that may be produced are:

A Workflow Model: This shows the sequence of activities in the process along with their inputs, outputs and dependencies. The activities in this model represent human actions.

A Dataflow or Activity Model: This represents the process as a set of activities each of which carries out some data transformation. It shows how the input to the process such as a specification is transformed to an output such as a design. The activities here may be at a lower level than activities in a workflow model. They may represent transformations carried out by people or by computers.

A Role/action Model: This represents the roles of the people involved in the software process and the activities for which they are responsible.

There are a number of different general models or paradigms of software development

The Waterfall Approach: This takes the above activities and represents them as separate process phases such as requirements specification, software design, implementation, testing and so on. After each stage is defined it is 'signed off' and development goes on to the following stage.

Evolutionary Development: This approach interleaves the activities of specification, development and validation. An initial system is rapidly developed from very abstract specifications. This is then refined with customer input to produce a system which satisfies the customer's needs. The system may then be delivered. Alternatively, it may be reimplemented using a more structured approach to produce a more robust and maintainable system.

Formal Transformation: This approach is based on producing a formal mathematical system specification and transforming this specification, using mathematical methods to a program. These transformations are 'correctness preserving' this means that you can be sure that the developed program meets its specification.

System Assembly from Reusable Components: This technique assumes that parts of the system already exist. The system development process focuses on integrating these parts rather than developing them from scratch.

2.3 Characteristics of a Software Model

When creating any form of software, the first question that comes to mind for each developer is, "What are the qualities that good software should have?" We'd want to state the basic expectations one has from any software before diving into technical qualities. First and foremost, a software solution must meet all of the customer's or end-needs. Users In addition, the software's development and maintenance costs should be modest. Software development should be finished within the stated time range.

Well these were the obvious things which are expected from any project (and software development is a project in itself). Now let's take a look at Software Quality factors. These set of factors can be easily explained by Software Quality Triangle. The three characteristics of good application software are:

1. Operational Characteristics
2. Transition Characteristic
3. Revision Characteristics

Operational Characteristics of a Software

These are functionality based factors and related to 'exterior quality' of software. Various Operational Characteristics of software are:

Correctness: The software which we are making should meet all the specifications stated by the customer.

Usability/Learnability: The amount of efforts or time required to learn how to use the software should be less. This makes the software user-friendly even for IT-illiterate people.

Integrity: Just like medicines have side-effects, in the same way software may have a side-effect i.e. it may affect the working of another application. But quality software should not have side effects.

Reliability: The software product should not have any defects. Not only this, it shouldn't fail while execution.

Efficiency: This characteristic relates to the way software uses the available resources. The software should make effective use of the storage space and execute command as per desired timing requirements.

Security: With the increase in security threats nowadays, this factor is gaining importance. The software shouldn't have ill effects on data/hardware. Proper measures should be taken to keep data secure from external threats.

Safety: The software should not be hazardous to the environment/life.

Revision Characteristics of Software

These engineering based factors of the relate to 'interior quality' of the software like efficiency, documentation and structure. These factors should be in-build in any good software. Various Revision Characteristics of software are:

Maintainability: Maintenance of the software should be easy for any kind of user.

Flexibility: Changes in the software should be easy to make.

Extensibility: It should be easy to increase the functions performed by it.

Scalability: It should be very easy to upgrade it for more work (or for more number of users).

Testability: Testing the software should be easy.

Modularity: Any software is said to made of units and modules which are independent of each other. These modules are then integrated to make the final software. If the software is divided into separate independent parts that can be modified, tested separately, it has high modularity.

Transition Characteristics of the Software

Interoperability: Interoperability is the ability of software to exchange information with other applications and make use of information transparently.

Reusability: If we are able to use the software code with some modifications for different purpose then we call software to be reusable.

Portability: The ability of software to perform same functions across all environments and platforms, demonstrate its portability.

Importance of any of these factors varies from application-to-application.

2.4 Software Development Life Cycle

The Software Development Life Cycle is process or method which is used to design, develop quality software. SDLC is a systematic process for building software that ensures the quality and correctness of the software built. SDLC process aims to produce high-quality software that meets customer expectations. The system development should be complete in the pre-defined time frame and cost. SDLC consists of a detailed plan which explains how to plan, build, and maintain specific software. Every phase of the SDLC life Cycle has its own process and deliverables that feed into the next phase.

ISO/IEC 12207 is an international standard for software life-cycle processes. It aims to be the standard that defines all the tasks required for developing and maintaining software.

SDLC Phase

SDLC consist different phases every phase of the SDLC life Cycle has its own process/ activities and it is linked with next phase or it provides the input to the next phase.

Requirement analysis

Feasibility study

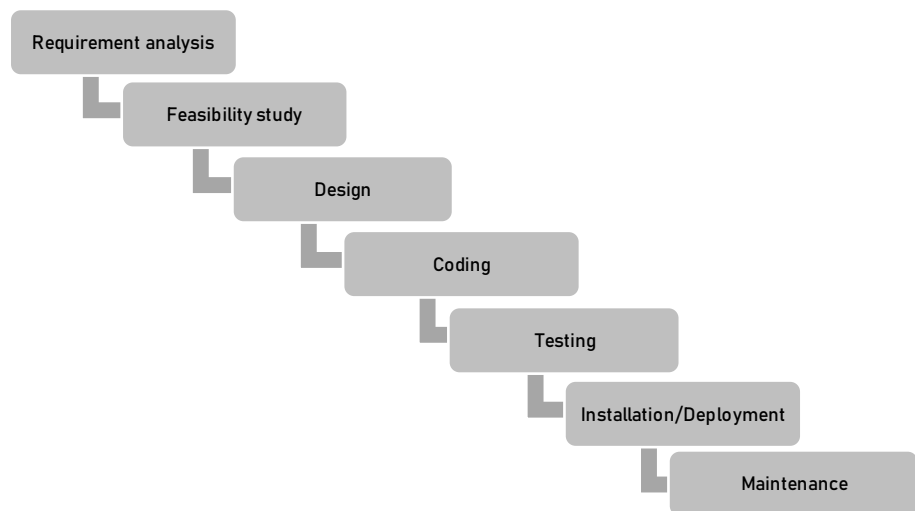
Design

Coding

Testing

Installation/Deployment

Maintenance



Requirement Analysis

Requirement Analysis is use to gather information from different stake holders e.g. sales department, market surveys and domain experts in the industry. This information is then used to plan the basic project approach and to conduct product feasibility study

Feasibility study

A feasibility study is simply an assessment of the practicality of a proposed plan or method.

Economic

Time

Operation feasibility

Technical

Legal

Design

The design phase helps define overall system architecture. In design phase documents are prepared as per the requirement specification document and feasibility study.

Coding

In this phase developers start build the entire system by writing code using the chosen programming language/ platform. In this phase tools use programming tools like compiler, interpreters, debugger to generate and implement the code

Testing

This phase is used to verify that the entire application works according to the customer requirement by using different testing techniques.

Unit Testing

Integration Testing

Regression Testing

Black box testing

White box testing

Beta Testing

Installation/Deployment

In this phase software is deployed to the production environment so users can start using the product. Based on the feedback given by the user's manager, the final software is released and checked for deployment issues. This phase allows users to safely play with the product before releasing it to the market.

Maintenance

This phase deals with the real world changing conditions, we need to update and advance the software to match.

Bug fixing

Upgrade

Enhancement

2.5 Prototype model

It is a software development model in which a prototype model is built, tested, and reworked until an acceptable prototype is achieved. It is a demo implementation of the actual product or system.

Prototyping is used to allow the users evaluate developer proposals and try them out before implementation. A prototype model usually exhibits limited functional capabilities, low reliability, and inefficient performance as compared to the actual software.

It is suitable where the project's requirements are not known in detail to customer. It is an iterative, trial and error method which takes place between developer and client.

Prototype model phase

Requirements gathering and analysis

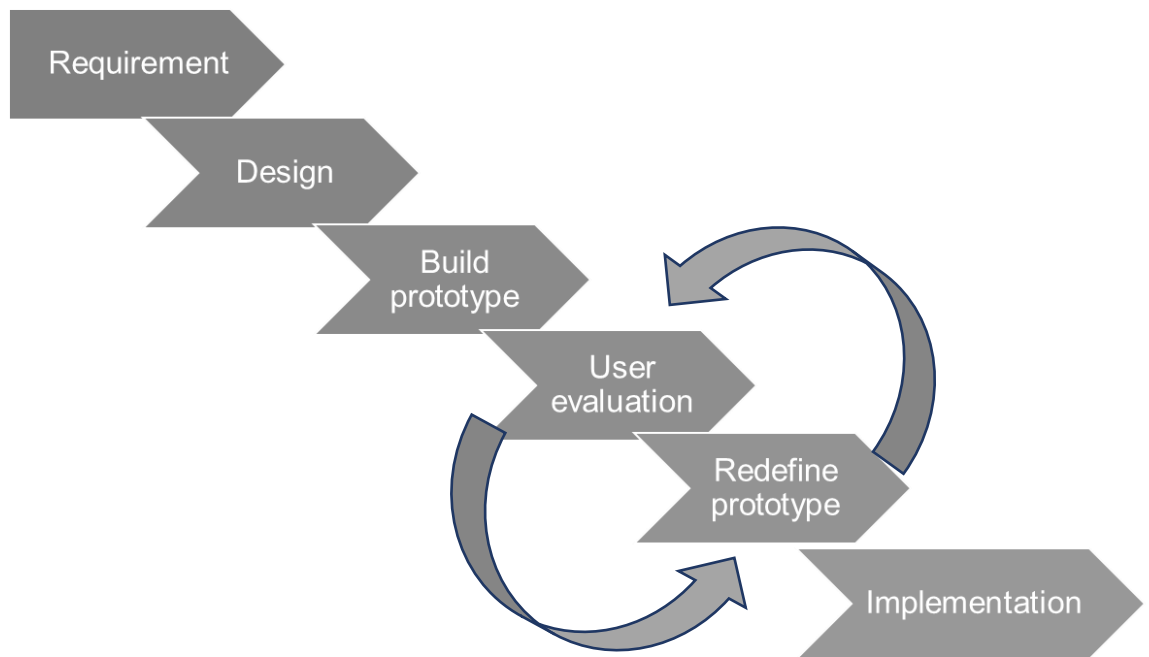
Design

Build prototype

User evaluation

Refining prototype

Implementation and Maintenance



Requirements gathering and analysis

The requirements of the system are defined. In this process, the users of the system are interviewed to know what their expectation from the system is. Use different method to gather information.

Design

In this phase, a simple design of the system is built. It gives a brief idea of the system to the user

Build prototype

In this phase the initial Prototype is developed, where the basic requirements are showcased and user interfaces are provided. The actual prototype is designed based on the information gathered from design phase

User evaluation

In this phase prototype developed is presented to the customer and the other important stakeholders in the project for an initial evaluation. The feedback is collected and used for further changes in the product under development. This phase help to find out the strength and weakness of the working model

Refining prototype

In this phase, Based upon customer feedback negotiations happen on factors like – time and budget constraints and technical feasibility of the actual implementation. Changes are accepted and incorporated in the new Prototype model and the cycle repeats until the customer expectations are met.

Implementation and Maintenance

After development of final system based on the final prototype, it is tested and deployed to production. The system undergoes routine maintenance and updates based upon real time environment changes for minimizing downtime and prevent large-scale failures and for.

Types of Prototyping Models

Rapid Throwaway prototypes

This technique offers a useful method of exploring ideas and getting customer feedback for each of them. In this method, a developed prototype need not necessarily be a part of the ultimately

accepted prototype. Customer feedback helps in preventing unnecessary design faults and hence, the final prototype developed is of better quality.

Evolutionary prototype

The prototype developed is incrementally refined based on customer's feedback until it is finally accepted. It helps you to save time as well as effort. That's because developing a prototype from scratch for every interaction of the process can sometimes be very frustrating.

Incremental Prototype

In incremental Prototyping, the final product is decimated into different small prototypes and developed individually. Eventually, the different prototypes are merged into a single product. This method is helpful to reduce the feedback time between the user and the application development team.

Extreme Prototype

Extreme prototyping method is mostly used for web development. It consists of three sequential phases.

- Basic prototype with all the existing page is present in the HTML format.
- You can simulate data process using a prototype services layer.
- The services are implemented and integrated into the final prototype.

Advantages of prototype model

- The Users involved in development. Errors can be detected in the initial stage of the software development process.
- Suitable where requirement are changing.
- Reduce Maintenance cost

Disadvantages of prototype model

- It is a slow and time taking process
- Cost increases with respect to time.
- Customer involvement is higher so it requirement changes are very dynamic and it impact overall product development

2.6 Water Fall Model

The waterfall model derives its name due to the cascading effect from one phase to the other as is illustrated in the figure below. In this model each phase is well defined, has a starting and ending point, with identifiable deliveries to the next phase.

- It is Linear sequential model
- Introduced by Winston W. Royce in 1970.
- The waterfall model is the classic lifecycle model.
- Whole system is developed in a sequential approach.
- Each phase is completed fully before the next begin.

Stages of a prototype model are:

Requirement analysis

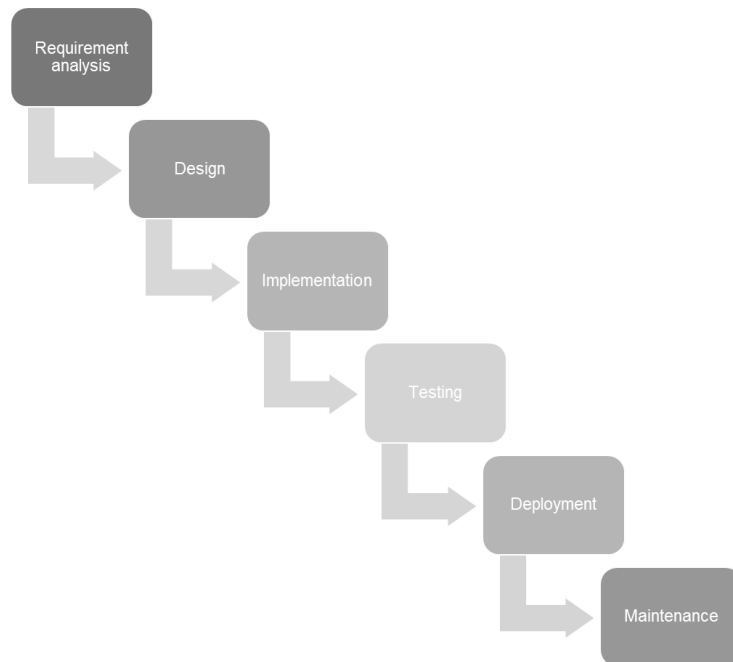
Design

Implementation

Testing

Deployment

Maintenance

**Requirement analysis**

- In this stage to understand the requirements of the system to be developed
- Software definition to be detailed and accurate with no ambiguities
- Documented requirements and reviewed.
- Software Requirement Specification (SRS) document is created

Design

In this phase the requirements gathered in the SRS transform into a suitable form for coding in a programming language or selection of platform. It helps in defining the overall system architecture.

Implementation

In this phase software design is translated into source code using any suitable programming language.

Testing

In this phase code is thoroughly examined based upon different testing methods.

- Unit testing
- Integration testing
- Alpha testing
- Beta testing

Deployment

After testing is done, the product is deployed in the customer environment or released into the market.

Maintenance

- It is performed by every user once the software has been delivered to the customer, installed, and operational as per customer request
- Meet the changing customer needs
- Adapted to accommodate changes in the external environment

Advantages

- Simple and easy to understand and use
- Phases in this model are processed one at a time
- Suited for smaller projects where requirements are well defined
- Clearly defined phase.
- Process and results are well documented

Disadvantages

- All requirements must be known upfront
- Not suitable where requirements are changing frequently
- Not a good model for complex and object-oriented projects.
- High amounts of risk and uncertainty
- It is difficult to measure progress within stages.

2.7 V model

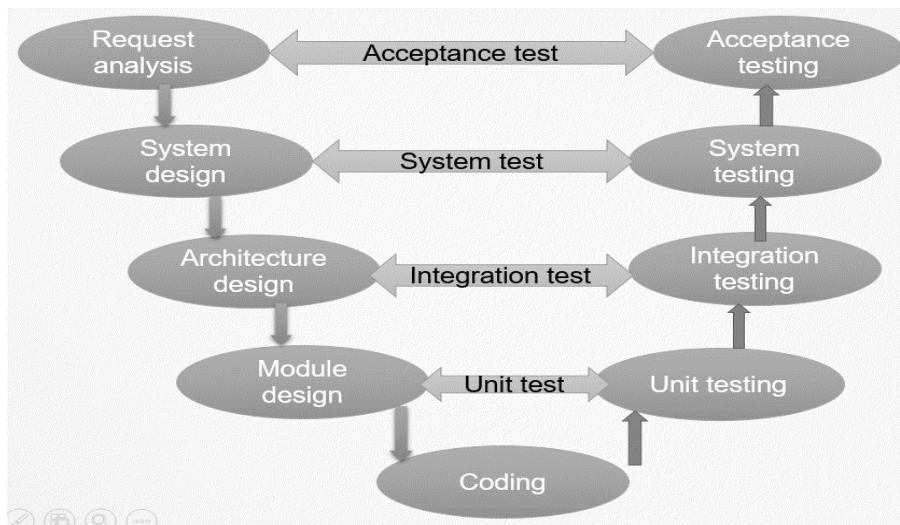
The V model is type of SDLC model. Testing phase parallel to each development phase, Process executes in a sequential manner in V-shape. It is also known as Verification and Validation model

The next phase starts only after completion of the previous phase i.e. for each development activity, there is a testing activity corresponding to it.

Verification and validation

Verification: Verification is a static analysis technique. In this technique, testing is done without executing the code. Examples include – Reviews, Inspection It is the process of evaluation of the product development phase to find whether specified requirements meet.

Validation: Validation is a dynamic analysis technique where testing is done by executing the code. Examples include functional and non-functional testing techniques.



Design phase

Requirement Analysis

System design

Architectural Design

Module design

Testing phase

Unit testing

Integration testing

System testing

User acceptance testing

Requirement Analysis

In this stage to understand the requirements of the system to be developed, this phase involves detailed communication with the customer and domain experts. Different method are used to gather information

System design

In this phase, a simple design of the system is built as per requirement gathered in previous phase. The system design will have the understanding and detailing the complete hardware and communication setup for the product under development.

Architectural Design

In architecture design it should understand all modules, brief functionality of each module, their interface relationships, dependencies, database tables, architecture diagrams, and technology detail

The system design is broken down further into modules taking up different functionality. This is also referred to as High Level Design (HLD).

Module design

In this phase, the system breaks down into small modules. The detailed internal design for all the system modules is specified, referred to as Low Level Design (LLD). In this phase design is checked for its compatibility with the other modules in the system architecture and the other external systems.

Coding

In this phase perform task based upon design phase inputs and follow guidelines for coding, Selection of programming language or platform.

Unit testing

In this phase Unit Test Plans (UTPs) are developed during the module design phase. These UTPs are executed to eliminate errors at code level or unit level. A unit is the smallest entity which can independently exist

Integration testing

In this phase modules are integrated and the system is tested. Integration testing is performed on the Architecture design phase. It verifies the communication of modules among themselves.

System testing

In this phase complete application is checked with its functionality, inter dependency, and communication. It tests the functional and non-functional requirements of the developed application. It is associated with the system design phase.

Acceptance testing

It is associated with the requirement analysis phase and involves testing the product in user environment. It uncovers the compatibility issues with the other systems available in the user environment.

Advantages

- This model focuses on verification and validation activities early in the life cycle
- Avoids the downward flow of the defects.
- This is a disciplined model and Phases are completed one at a time.
- Useful in small projects where requirements are clear

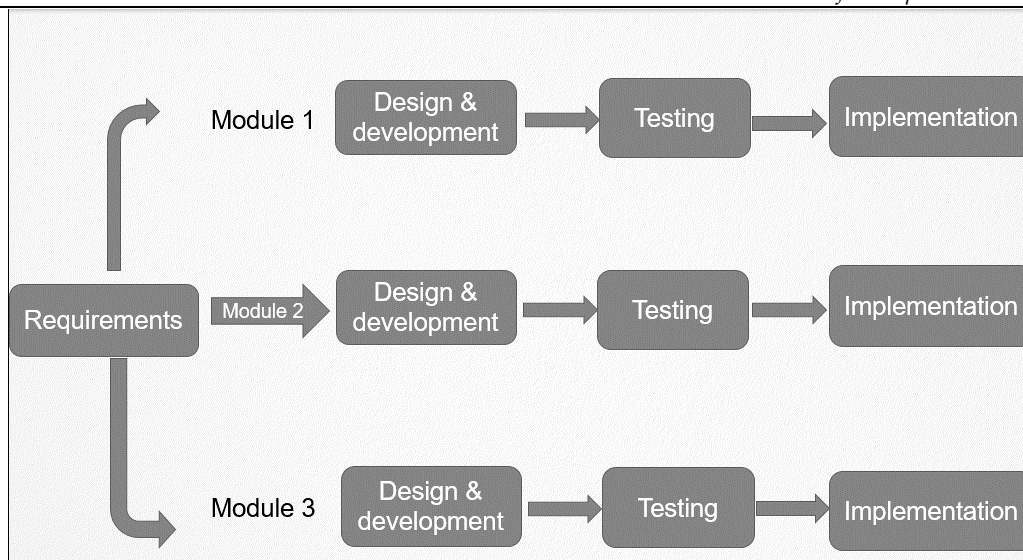
Disadvantages

- It is not suitable for projects where requirements are more dynamic and contains high risk of changing
- Not suitable for complex projects.

2.8 Incremental Model

In Incremental Model requirements are broken down into multiple standalone modules of software development cycle. Multiple development cycles take place here, making the life cycle a “multi-waterfall” cycle.

Each iteration passes through the requirements, design, coding and testing phases. After completion of one module the system adds function to the previous release until all designed functionality has been implemented.



Incremental Model Phase

- Requirement analysis
- Design & Development
- Testing
- Implementation

Requirement analysis

In this phase requirements are gathered and analyzed using different techniques and tools. This phase involves detailed communication with the customer and domain experts.

Design and Development

In this phase the design of the system functionality and the development method are finished with success. When software develops new practicality, the incremental model uses style and development phase.

Testing

In this phase the testing phase checks the performance of each existing function as well as additional functionality. Different methods and tools are used to test the behavior of each task.

Implementation

In this phase final coding is done based upon designing and development phase and tests the functionality in the testing phase. After completion of this phase, the number of the product working is enhanced and upgraded up to the final system product.

Characteristics of incremental model

- System development is broken down into many mini development projects
- Partial systems are successively built to produce a final total system
- Highest priority requirement is tackled first
- Once the incremented portion is developed, requirements for that increment are frozen

Advantages

- Easy to Identify error
- Flexibility is high, less expensive to change requirements and scope
- Easier to test and debug
- Design and development is easy as modules are handled individually.

Disadvantages

- Good planning is required
- Problems might cause due to all requirements are handled in different modules.
- Rectifying a problem in one unit requires correction in all the units and consumes a lot of time

2.9 Agile method of software development

Agile Methodology

Agility is the ability to both create and respond to change in order to profit in a turbulent business environment. It is a practice that promotes continuous iteration of development and testing throughout the software development lifecycle of the project.

Agility is the ability to both create and respond to change in order to profit in a turbulent business environment. It is a practice that promotes continuous iteration of development and testing throughout the software development lifecycle of the project.

Agility is not merely reaction, but also action. It is ability to react, to respond quickly and effectively to both anticipated and unanticipated changes in the business environment. Agility means being responsive or flexible within a defined context.

Agile vs. Traditional Models

Agile model is based on the adaptive software development methods. Traditional SDLC models like the waterfall model are based on a predictive approach.

Predictive methods entirely depend on the requirement analysis and planning done in the beginning of cycle. Agile uses an adaptive approach where there is no detailed planning and there is clarity on future tasks only in respect of what features need to be developed.

Agile Process Model

It is a combination of iterative and incremental process models with focus on process adaptability and customer satisfaction by rapid delivery of working software product. Agile Methods break the product into small incremental builds. These builds are provided in iterations.

Agile process model can be implemented with the help of various methods:

- Extreme Programming
- DSDM
- Essential Unified Process
- Agile Data Method
- Agile Modeling
- Agile Unified Process

Advantages of the Agile Models

- Promotes teamwork and cross training.
- Realistic approach to software development.
- Functionality can be developed rapidly and demonstrated.
- Resource requirements are minimum.
- Suitable for fixed or changing requirements
- Delivers early partial working solutions.

Disadvantages of the Agile Models

- Not suitable for complex problem
- More risk of sustainability, maintainability and extensibility.
- Depends heavily on customer interaction.
- Strict delivery management dictates the scope, functionality to be delivered, and adjustments to meet the deadlines.

Summary

- A software process is a set of activities, together with ordering constraints among them, such that if the activities are performed properly and in accordance with the ordering constraints, the desired result is produced.
- Software Process framework is a set of guidelines, concepts and best practices that describes high level processes in software engineering.
- The sequence of activities specified by the process is typically at an abstract level because they have to be usable for a wide range of projects.
- A software process model is a simplified description of a software process, which is presented from a particular perspective.

- In waterfall model each phase is well defined, has a starting and ending point, with identifiable deliveries to the next phase.
- In a prototype model, a working prototype is built with the available set of requirements such that it has limited functionalities, low reliability and performance.
- An iterative lifecycle model does not attempt to start with a full specification of requirements.

Keywords

Workflow Model: This shows the sequence of activities in the process along with their inputs, outputs and dependencies.

Dataflow or Activity Model: This represents the process as a set of activities each of which carries out some data transformation.

Design Phase: A Design phase is a phase in which a software solution to meet the requirements is designed.

Prototype Model: It is a model in which a working prototype is built with the available set of requirements such that it has limited functionalities, low reliability and performance.

Requirements Phase: A Requirements phase is a phase in which the requirements for the software are gathered and analyzed.

Software Processes: Software processes are the activities involved in producing and evolving a software system.

Software Process Framework: It is a set of guidelines, concepts and best practices that describes high level processes in software engineering.

Software Process Model: A software process model is a simplified description of a software process, which is presented from a particular perspective

Self Assessment

1. Which is not a SDLC phase?

- A. Design
- B. Coding
- C. Testing
- D. Coupling

2. Which is not a part of feasibility study?

- A. Time
- B. Operational
- C. Module
- D. Cost

3. Which is not a testing type?

- A. Unit
- B. Beta
- C. Cohesion
- D. Black box

4. Prototype model is_____

- A. Suitable where the project's requirements are not known in detail to customer
- B. It is a demo implementation of the actual product or system
- C. Used to allow the users evaluate developer proposals
- D. All of above

5. Which is not a part of prototype model phases?

- A. Requirement
- B. Design
- C. Process
- D. Implementation

6. Which is not a prototype model type?

- A. Evolutionary
- B. Beta
- C. Incremental
- D. Extreme

7. Waterfall model is_____

- A. It is linear sequential model
- B. It is the classic lifecycle model
- C. Introduced by Winston W. Royce in 1970
- D. All of above

8. Stages in Waterfall model are_____

- A. Design
- B. Implementation
- C. Testing and Deployment
- D. All of above

9. Information is gathered in _____ phase

- A. Design
- B. Coding
- C. Requirement analysis
- D. Maintenance

10. Incremental Model is_____

- A. Broken down into multiple standalone modules of software development cycle.
- B. Multiple development cycles take place
- C. Each iteration passes through the requirements, design, coding and testing phases
- D. All of above

Answer for Self Assessment

- | | | | | |
|------|------|------|------|-------|
| 1. D | 2. C | 3. C | 4. D | 5. C |
| 6. B | 7. D | 8. D | 9. C | 10. D |

Review Questions

1. Explain the concept of software processes.
2. Describe the relation between Processes, Projects and Products.
3. Explain the concept of software process model with examples.
4. Discuss the number of different general models or paradigms of software development.
5. Discuss the characteristics of a Software Model.

6. Explain the working of waterfall model.
7. Describe the phases included in iterative model.
8. What is Agile Methodology?
9. Name few Agile Process Model.



Further Reading

- Books Rajib Mall, Fundamentals of Software Engineering, 2nd Edition, PHI.
- Richard Fairpy, Software Engineering Concepts, Tata McGraw Hill, 1997.
- R.S. Pressman, Software Engineering – A Practitioner’s Approach, 5th Edition, Tata McGraw Hill Higher education.
- Sommerville, Software Engineering, 6th Edition, Pearson Education.



- <http://www.ijcsi.org/papers/7-5-94-101.pdf>
- <http://people.sju.edu/~jhodgson/se/models.html>
- <http://www.ics.uci.edu/~wscacchi/Papers/SE-Encyc/Process-Models-SEEncyc.pdf>
- <https://www.tutorialspoint.com/>
- <https://www.guru99.com/>